

WS63 uapi_uart_* 驱动接口说明

适用平台：九联星闪开发板，基于海思 WS63E 芯片。

本文档基于 SDK 头文件 `device/soc/hisilicon/ws63v100/sdk/include/driver/uart.h`，归纳海思 WS63 侧 **UART 统一应用接口（uapi_uart_*）** 的职责、数据结构与典型用法，便于在 LiteOS / OpenHarmony 应用或中间件中直接调用底层驱动。

文档更新于 2026-05-06。

一、接口在软件栈中的位置

层次	说明
应用 / 组件	可使用 <code>printf</code> （标准输出经系统移植调试口），或经 OpenHarmony <code>**IoTuart**</code> （ <code>base/iothardware</code> ）访问 UART。
<code>**uapi_uart_*</code> (本文)**	SDK driver 层对外 C API，封装 HAL、中断、可选 DMA，是 IoT UART HAL 在 WS63 上最终调用的能力之一（见 <code>device/soc/hisilicon/ws63v100/adapters/hals/iot_hardware/wifi/wifi_lite/hal_iot_uart.c</code> ）。
HAL / Porting	<code>hal_uart_*</code> 、 <code>uart_porting.*</code> ：引脚、时钟、中断向量、DMA 握手等与芯片绑定。

`uart.h` 在编译期依赖 `**CONFIG_UART_SUPPORT_TX / CONFIG_UART_SUPPORT_RX**` 等宏；若二者均未定义会直接 `#error`。

二、头文件与总线编号

- 主头文件：**`device/soc/hisilicon/ws63v100/sdk/include/driver/uart.h`
- 间接依赖：**`errcode.h`、`hal_uart.h`、`uart_porting.h`；总线类型 `**uart_bus_t**` 定义于芯片平台头（如 `platform_core.h`），典型取值为 `**UART_BUS_0`、`UART_BUS_1`、`UART_BUS_2**`（具体数量由 `UART_BUS_MAX_NUMBER` 决定），`UART_BUS_NONE` 表示无效。

九联星闪 / WS63 工程中常见约定：**调试/日志 UART** 与 `**CONFIG_DEBUG_UART`、`CONFIG_LOG_UART**` 等配置相关，使用某一路 `UART_BUS_X` 前需对照原理图与本产品 `mconfig` / 引脚说明，避免与系统已占用的调试口冲突。

三、核心数据结构

3.1 引脚与线路属性

类型	含义
<code>uart_pin_config_t</code>	等同 <code>hal_uart_pin_config_t</code> ： <code>tx_pin</code> 、 <code>rx_pin</code> 、 <code>cts_pin</code> 、 <code>rts_pin</code> （无硬件流控时常用 <code>**PIN_NONE**</code> ）。
<code>uart_attr_t</code>	等同 <code>hal_uart_attr_t</code> ： <code>baud_rate</code> 、 <code>data_bits</code> 、 <code>stop_bits</code> 、 <code>parity</code> 、 <code>flow_ctrl</code> 。数据位/停止位/校验/流控枚举见 <code>hal_uart.h</code> （如 <code>UART_DATA_BIT_8</code> 、 <code>UART_STOP_BIT_1</code> 、 <code>UART_PARITY_NONE</code> 、 <code>UART_FLOW_CTRL_NONE</code> ）。

类型	含义
<code>uart_extra_attr_t</code>	扩展属性：是否启用 TX/RX DMA 、 TX/RX FIFO 中断阈值 等；是否走 DMA 与后续应调用 <code>**uapi_uart_write**</code> 还是 <code>**uapi_uart_write_by_dma**</code> 等一致（头文件注释中有对应说明）。
<code>uart_buffer_config_t</code>	接收缓存 ： <code>rx_buffer</code> 指针 + <code>rx_buffer_size</code> 。驱动用其作为 RX 侧工作区； 初始化时必须提供合法缓冲区 （除非产品裁剪为仅 TX）。

3.2 发送相关（条件编译）

在定义了 `CONFIG_UART_SUPPORT_TX` 时：

- `**uart_tx_callback_t`：`uapi_uart_write_int` **发送完成回调**，在中断上下文执行**。
- 若同时支持 DMA：`uart_write_dma_config_t`（源/目的宽度、burst、优先级等）用于 `uapi_uart_write_by_dma` / `uapi_uart_read_by_dma`。

3.3 接收条件与回调（条件编译）

在定义了 `CONFIG_UART_SUPPORT_RX` 时：

- `**uart_rx_condition_t**`（与 `size` 参数配合）：例如「FIFO 满或线路空闲」「满或达到指定长度」等组合（`UART_RX_CONDITION_FULL_OR_IDLE`、`UART_RX_CONDITION_FULL_OR_SUFFICIENT_DATA`、`UART_RX_CONDITION_FULL_OR_SUFFICIENT_DATA_OR_IDLE`）。
- `**uart_rx_callback_t**`：`uapi_uart_register_rx_callback` 注册的回调，**在中断上下文执行**；注释说明回调执行期间新到的数据可能被丢弃，回调内应快速处理或只做拷贝/投递。
- `**uart_error_callback_t**`：帧错误、奇偶校验错误、溢出等统一形态的错误回调类型。

3.4 中断 RX 回调：单次 `length` 常为约 16 字节（WS63 / UART v151）

在 `**CONFIG_UART_SUPPORT_PORTTING_IRQ**` 打开、走 `**uapi_uart_register_rx_callback**` 路径时，回调参数 `**length**` 表示本次交给应用的数据字节数。在 **WS63 当前 HAL（hal_uart_v151）与默认 mconfig 下**，常见现象是：**单次回调里 `length` 往往不超过约 16**，更长的一帧会**分多次**回调。原因如下（便于对照源码，非应用层配置项）。

1. **RX FIFO 中断门限**：`hal_uart_v151.c` 中 `hal_uart_v151_init` 对 RX 使用 `**HAL_UART_RX_FIFO_AVAILABLE_LEVEL_1_4`（1/4 满）** 初始化 FIFO 中断；`UART_CTRL_GET_RX_FIFO_THRESHOLD` 在 1/4 档位返回 `**CONFIG_UART_FIFO_DEPTH >> 2`。**产品侧常见 `**CONFIG_UART_FIFO_DEPTH` 为 64，故阈值为 16**（见 `device/soc/hisilicon/ws63v100/sdk/drivers/drivers/hal/uart/v151/hal_uart_v151.c` 中 `hal_uart_v151_get_uart_rxfifo_threshold`）。
2. `**register_rx_callback` 的 `size` 会被截断：`uart.c` **中在未定义 `**CONFIG_UART_NOT_SUPPORT_RX_CONDITON_SIZE_OPTIMIZE` 时**，会把注册的 `**size**` 限制为 **不超过上述 RX FIFO 阈值**（`hal_uart_ctrl(..., UART_CTRL_GET_RX_FIFO_THRESHOLD, ...)` 后再取 `min`）。因此应用即使传入较大 `**size`，条件尺寸**也不会超过该阈值**；这与「**单次回调里看到的字节数**」相关，但不等同：**真正进入回调时的 `**length` 仍由 当次中断里 ISR 从硬**

- 件 FIFO 搬入 `rx_buffer` 的字节数决定，通常与 FIFO 深度/触发时机 同量级，故实验中常见 ≤ 16 一批。
3. 应用层能否「改掉 16」：仅通过 `uart.h` 公开 API 无法修改 HAL 的 RX FIFO 门限；`uart.h` 未暴露 `UART_CTRL_SET_RX_FIFO_LEVEL` 一类设置。若业务需要逻辑上长于 16 的报文，应在任务中对多次回调数据做组包/状态机；若必须在硬件/驱动层增大单次上报粒度，需改 SDK HAL（例如调整 `hal_uart_init_fifo` 的 RX level，并评估溢出与实时性）或使用 DMA 等路径（见本文 9.4）。

四、`uapi_uart_*` 函数一览（按功能）

以下函数名与语义均以 `uart.h` 为准；部分接口受 `**CONFIG_UART_SUPPORT_*` 控制，未开启时无声明。

4.1 生命周期与属性

函数	作用
<code>uapi_uart_init</code>	初始化指定 <code>bus</code> ：引脚、基本/扩展属性、RX 缓冲配置。成功返回 <code>ERRCODE_SUCC</code> 。
<code>uapi_uart_deinit</code>	去初始化，释放驱动占用的资源。
<code>uapi_uart_set_attr</code> / <code>uapi_uart_get_attr</code>	运行时修改或查询 <code>uart_attr_t</code> （波特率、数据位等）。
<code>uapi_uart_has_pending_transmissions</code>	是否仍有未完成发送。
<code>uapi_uart_rx_fifo_is_empty</code> / <code>uapi_uart_tx_fifo_is_empty</code>	查询 RX/TX FIFO 是否空。

4.2 轮询方式收发（最常用）

函数	作用
<code>uapi_uart_write</code>	轮询/阻塞式发送： <code>buffer</code> 、 <code>length</code> 、 <code>**timeout**</code> ；返回实际写入字节数（ <code>int32_t</code> ）。
<code>uapi_uart_read</code>	从 UART 读取至多 <code>length</code> 字节到 <code>buffer</code> ，带 <code>**timeout**</code> ；返回实际读到的长度。

注意：头文件中 `uapi_uart_read` 的 `buffer` 参数类型为 `const uint8_t *`，与常见「写入缓冲区」语义相反，属于 SDK 签名风格；实际用法为传入可写缓冲区指针（与工程内 `test_suite_uart`、`hal_iot_uart` 等调用方式一致）。

4.3 中断方式发送

函数	作用
<code>uapi_uart_write_int</code>	中断发送，完成后调用 <code>uart_tx_callback_t</code> ；回调在中断里，不宜做耗时操作或阻塞。

4.4 DMA 方式（需 CONFIG_UART_SUPPORT_DMA）

函数	作用
uapi_uart_write_by_dma / uapi_uart_read_by_dma	DMA 发送/接收，需正确填写 uart_write_dma_config_t。
uapi_uart_register_read_by_dma_callback 等	在 CONFIG_UART_SUPPORT_INT_TRIGGER_DMA 下， RX 中断触发 DMA 搬运的注册/去注册。

使用前须在 `**uart_extra_attr_t` 中打开对应 TX/RX DMA 开关，并与 HAL/时钟/DMA 通道配置一致。

4.5 接收回调与错误注册

函数	作用
uapi_uart_register_rx_callback / uapi_uart_unregister_rx_callback	按 条件 + size 注册 RX 完成回调。
uapi_uart_register_parity_error_callback	奇偶校验错误。
uapi_uart_register_frame_error_callback	帧错误。
uapi_uart_register_overrun_error_callback	溢出错误。
uapi_uart_flush_rx_data	清空驱动 RX 缓冲中的 数据。
uapi_uart_update_rx_buff	更换 RX 缓冲地址与长 度。

4.6 低功耗（需 CONFIG_UART_SUPPORT_LPM）

函数	作用
uapi_uart_suspend / uapi_uart_resume	挂起/恢复 UART 通道（参数 arg 由平台定 义）。

五、典型使用流程

5.1 仅发送（轮询）

- 1. 准备 `uart_pin_config_t`（TX/RX 引脚，CTS/RTS 可 `PIN_NONE`）。
- 2. 填写 `uart_attr_t`（如 115200、8N1、无流控）。
- 3. 若 `uapi_uart_init` 要求 RX 缓冲：提供静态数组，填 `uart_buffer_config_t`（即使当前只发不收，部分产品仍要求非空缓冲，以实际 SDK 行为为准）。
- 4. `extra_attr`：关闭 DMA 时使用全 0 或显式 `tx_dma_enable/rx_dma_enable` 为 false（以工程模板为准）。
- 5. 调用 `**uapi_uart_init(bus, pins, attr, extra_attr, &buf_cfg)**`。

6. 循环或按需调用 `**uapi_uart_write(bus, data, len, timeout)**`。

7. 结束前 `**uapi_uart_deinit(bus)**`。

5.2 接收：轮询读

在 `init` 成功后周期性或阻塞调用 `**uapi_uart_read(bus, buf, len, timeout)**`，根据返回值处理粘包/半包。

5.3 接收：中断回调

1. `uapi_uart_init` 同上，保证 **RX 缓冲** 足够大。
2. `**uapi_uart_register_rx_callback(bus, condition, size, my_rx_cb)**`。
3. 在 `my_rx_cb` 内只做简短处理（例如拷贝到队列），**禁止** `printf` 过重、长时间锁、阻塞 API。

WS63 注意：单次 RX 回调的 `**length` 常见约 **16 字节上限**（与 FIFO 阈值有关），长帧需多次回调后在任务里拼接；详见 **第三节 3.4**。

六、与 OpenHarmony IoTUart* 的对应关系（便于选型）

应用若写在 **OpenHarmony iothardware** 之上，通常调用 `**IoTUartInit / IoTUartWrite / IoTUartRead**` 等；WS63 适配在 `**hal_iot_uart.c**` 中把上述 IoT 接口映射到 `**uapi_uart_init、uapi_uart_write、uapi_uart_read**`（及 DMA 分支等）。

- **教学/快速验证：**`printf` 或 `uart_printf` 示例即可。
- **指定波特率、指定 UART 口、与模块对接：**优先 `**IoTUart**`（与系统配置一致），或直接 `**uapi_uart_***`（需自行处理引脚与缓冲，适合驱动级或 SDK 内模块）。

七、注意事项小结

1. **中断上下文：**`uart_rx_callback_t`、`uart_tx_callback_t`、错误类回调均在**中断**中执行，与 GPIO/RTOS 文档中「中断里少做事」原则一致。
 2. **RX 缓冲：**`uart_buffer_config_t` 由调用方分配生命周期，**初始化期间**必须有效；缓冲区过小易导致丢包或频繁触发「满」条件。
 3. **中断 RX 单次 length (WS63)：**默认 HAL 下 RX 回调**常按约 16 字节一批**触发语义，**不是**应用把 `rx_buffer_size` 设大就能单次收满；长数据应**多次回调组包**或改用轮询/DMA，说明见 **3.4**。
 4. **超时参数：**`uapi_uart_write / uapi_uart_read` 的 `timeout` 单位与取值以 SDK 实现及同仓库示例为准（如测试套件中的用法）。
 5. **DMA 与 extra_attr：**启用 DMA 后应使用 `**uapi_uart_*_by_dma**` 路径，并配置 DMA 握手与通道。
 6. **多 UART：**`UART_BUS_0` 常被调试/日志占用，应用层另开端口时务必核对 **引脚复用**（`uart_port_config_pinmux` 等 porting 行为）。
-

八、源码与参考路径

内容	路径
<code>**uapi_uart_*</code> 声明**	<code>device/soc/hisilicon/ws63v100/sdk/include/driver/uart.h</code>
实现	<code>device/soc/hisilicon/ws63v100/sdk/drivers/drivers/driver/uart/uart.c</code>
引脚/中断 porting	<code>device/soc/hisilicon/ws63v100/sdk/drivers/chips/ws63/porting/uart/uart_porting.c</code> 、 <code>uart_porting.h</code>
HAL 类型与枚举	<code>device/soc/hisilicon/ws63v100/sdk/drivers/drivers/hal/uart/hal_uart.h</code>
总线枚举	<code>device/soc/hisilicon/ws63v100/sdk/drivers/chips/ws63/include/platform_core.h</code> (<code>uart_bus_t</code>)
OpenHarmony 适配	<code>device/soc/hisilicon/ws63v100/adapters/hals/iot_hardware/wifi/wifi_lite/hal_iot_uart.c</code>
调用示例	<code>device/soc/hisilicon/ws63v100/sdk/test/common/testsuite/src/test_suite_uart.c</code> 、 <code>sdk/application/samples/peripheral/uart/uart_demo.c</code>

九、三种方式的读写代码示例

以下示例为**教学向精简代码**，突出三种模式的 API 组合；**引脚宏**（如 `S_MGPI00`）、**总线号**（如 `UART_BUS_1`）及 `**data_bits` 数值与枚举的映射**需以当前产品 `uart_demo`、`hal_iot_uart.c` 及 `hal_uart.h` 为准。完整可编译工程可参考 `sdk/application/samples/peripheral/uart/uart_demo.c`（通过 `CONFIG_UART_POLL_TRANSFER_MODE` / `CONFIG_UART_INT_TRANSFER_MODE` / `CONFIG_UART_DMA_TRANSFER_MODE` 切换模式）。

9.1 公共准备（三种方式共用）

```
#include "uart.h"
#include "pinctrl.h"    /* 若需 uapi_pin_set_mode 配置复用，按产品宏使用 */

#define MY_UART_BUS      UART_BUS_1    /* 勿占用系统调试口时自行选择 */
#define RX_BUF_SIZE      128

static uint8_t s_rx_buf[RX_BUF_SIZE];

static void uart_common_hw_init(void)
{
    uart_pin_config_t pins = {
        .tx_pin = S_MGPI00,    /* 按原理图与 SDK 宏替换 */
        .rx_pin = S_MGPI01,
        .cts_pin = PIN_NONE,
        .rts_pin = PIN_NONE,
    };

    uart_attr_t attr = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_BIT_8,    /* 若工程用数值 3 表示 8 位，与 uart_demo 保持一致 */
        .stop_bits = UART_STOP_BIT_1,
        .parity = UART_PARITY_NONE,
        .flow_ctrl = UART_FLOW_CTRL_NONE,
    };
}
```

```

uart_buffer_config_t buf_cfg = {
    .rx_buffer = s_rx_buf,
    .rx_buffer_size = RX_BUF_SIZE,
};

/* 轮询模式: extra_attr 可为 NULL */
(void)uapi_uart_init(MY_UART_BUS, &pins, &attr, NULL, &buf_cfg);
}

```

9.2 轮询方式 (uapi_uart_read / uapi_uart_write)

特点: 在任务上下文中**阻塞或超时等待**收发完成, 实现简单; CPU 占用相对较高。

```

#define POLL_TIMEOUT 0 /* 与 uart_demo 一致时可填 0; 具体含义以 SDK 实现为准 */

void uart_poll_echo_once(void)
{
    int32_t n;

    n = uapi_uart_read(MY_UART_BUS, s_rx_buf, 16, POLL_TIMEOUT);
    if (n > 0) {
        (void)uapi_uart_write(MY_UART_BUS, s_rx_buf, (uint32_t)n, POLL_TIMEOUT);
    }
}

```

发送固定数据:

```

void uart_poll_send_hello(void)
{
    const uint8_t msg[] = "hello\r\n";
    (void)uapi_uart_write(MY_UART_BUS, msg, sizeof(msg) - 1, POLL_TIMEOUT);
}

```

9.3 中断方式 (接收回调 + uapi_uart_write_int)

特点: **接收**通过 `uapi_uart_register_rx_callback` 在**中断**里得到数据; **发送**用 `uapi_uart_write_int`, 完成时在 `**uart_tx_callback_t` 中通知。回调内勿做耗时操作, 可置标志位后在任务中续处理。

WS63 / v151: 单次 `**uart_rx_isr_cb` 的 `length` 常见**不超过约 16** (RX FIFO 1/4 满阈值、`CONFIG_UART_FIFO_DEPTH` 默认 64 时为 16; `uart.c` 还会将注册的 `**size` 截断到该阈值)。协议帧更长时请在任务中**拼接**多次回调数据, 或评估 **DMA** / 修改 HAL FIFO 门限 (非 `uart.h` 应用 API)。详见 **第三节 3.4**。

初始化需在非 DMA 场景下 `extra_attr` 为 `NULL` (或关闭 DMA), 与轮询相同; 随后注册 RX 回调:

```

#include <stdbool.h>
#include "securec.h"

static volatile int s_rx_ready;

```

```

static void uart_rx_isr_cb(const void *buffer, uint16_t length, bool error)
{
    if (error || buffer == NULL || length == 0) {
        return;
    }
    /* 快速拷贝到应用缓冲; length 不超过 s_rx_buf 时可用 */
    if (length <= RX_BUF_SIZE) {
        (void)memcpy_s(s_rx_buf, RX_BUF_SIZE, buffer, length);
    }
    s_rx_ready = 1;
}

static void uart_tx_done_cb(const void *buffer, uint32_t length, const void
*params)
{
    (void)buffer;
    (void)length;
    (void)params;
    /* 发送完成: 可置位或释放缓冲区 (注意: 中断上下文) */
}

void uart_int_mode_setup(void)
{
    uart_common_hw_init();

    /* 收到 ≥1 字节且满足 FIFO 满/空闲等条件时触发, size 含义见 uart.h */
    (void)uapi_uart_register_rx_callback(MY_UART_BUS,
        UART_RX_CONDITION_FULL_OR_SUFFICIENT_DATA_OR_IDLE,
        1,
        uart_rx_isr_cb);
}

void uart_int_send_in_task(void)
{
    const uint8_t reply[] = "ACK\r\n";

    if (uapi_uart_write_int(MY_UART_BUS, reply, sizeof(reply) - 1,
        NULL, uart_tx_done_cb) != ERRCODE_SUCC) {
        /* 发送入队失败 */
    }
}

```

任务中轮询 `s_rx_ready` 后再调用 `uart_int_send_in_task()` 即为常见 **中断收 + 任务发** 模型。

9.4 DMA 方式 (`uapi_uart_write_by_dma / uapi_uart_read_by_dma`)

特点: 大批量搬运由 DMA 完成, 适合吞吐要求高、包长较大的场景。需满足:

1. 编译打开 `**CONFIG_UART_SUPPORT_DMA**`, 并通常调用 `**uapi_dma_init / uapi_dma_open**` (以 `hal_iot_uart.c`、`uart_demo.c` 为准)。

2. `**uart_extra_attr_t**` 中 `**tx_dma_enable / rx_dma_enable**` 置为 `true`，并在 `**uapi_uart_init**` 时传入该结构体（不能为 `NULL`）。
3. 使用 `**uart_write_dma_config_t**` 填写宽度、burst、优先级等，与 SDK 示例保持一致。

```
#if defined(CONFIG_UART_SUPPORT_DMA)
#include "dma.h"

static uart_write_dma_config_t s_dma_cfg = {
    .src_width = 0,          /* 0: 1 byte, 见 uart.h 注释 */
    .dest_width = 0,
    .burst_length = 2,       /* 与 uart_demo / hal_iot_uart 常用值一致 */
    .priority = 0,
};

static void uart_dma_hw_init(void)
{
    uart_pin_config_t pins = {
        .tx_pin = S_MGPI00,
        .rx_pin = S_MGPI01,
        .cts_pin = PIN_NONE,
        .rts_pin = PIN_NONE,
    };
    uart_attr_t attr = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_BIT_8,
        .stop_bits = UART_STOP_BIT_1,
        .parity = UART_PARITY_NONE,
        .flow_ctrl = UART_FLOW_CTRL_NONE,
    };
    uart_extra_attr_t extra = {
        .tx_dma_enable = true,
        .tx_int_threshold = 0,
        .rx_dma_enable = true,
        .rx_int_threshold = 0,
    };
    uart_buffer_config_t buf_cfg = {
        .rx_buffer = s_rx_buf,
        .rx_buffer_size = RX_BUF_SIZE,
    };

    uapi_dma_init();
    uapi_dma_open();
    (void)uapi_uart_init(MY_UART_BUS, &pins, &attr, &extra, &buf_cfg);
}

void uart_dma_transfer_chunk(uint32_t len)
{
    int32_t r;

    if (len > RX_BUF_SIZE) {
        len = RX_BUF_SIZE;
    }
    r = uapi_uart_read_by_dma(MY_UART_BUS, s_rx_buf, len, &s_dma_cfg);
    if (r == (int32_t)len) {
```

```
        (void)uapi_uart_write_by_dma(MY_UART_BUS, s_rx_buf, len, &s_dma_cfg);
    }
}
#endif /* CONFIG_UART_SUPPORT_DMA */
```

若工程未定义 `CONFIG_UART_SUPPORT_DMA`，上述接口在 `uart.h` 中不可用，请仍使用轮询或中断方式。

9.5 三种方式对照

方式	主要 API	适用场景
轮询	<code>uapi_uart_read</code> 、 <code>uapi_uart_write</code>	短数据、逻辑简单、对实时性要求不极端
中断	<code>uapi_uart_register_rx_callback</code> 、 <code>uapi_uart_write_int</code>	需异步收包、避免长阻塞读； WS63 下单次 RX 回调 <code>length</code> 常见约 ≤16 ，长帧须组包 (3.4)
DMA	<code>uapi_uart_read_by_dma</code> 、 <code>uapi_uart_write_by_dma</code> + <code>extra_attr</code> 开 DMA	较长帧、高吞吐、需降低 CPU 参与

实验教学可与 [08-UART实验指导书.md](#) 中 `printf` 实验对照阅读：前者偏应用输出，本文偏 **底层** `uapi` 直接控串口。